



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 15 de enero
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	14
Apéndice B Especificación de Requisitos	15
B.1. Introducción	15
B.2. Objetivos generales	15
B.3. Catálogo de requisitos	15
B.4. Especificación de requisitos	18
Apéndice C Especificación de diseño	21
C.1. Introducción	21
C.2. Diseño de datos	21
C.3. Diseño arquitectónico	21
C.4. Diseño procedimental	21
Apéndice D Documentación técnica de programación	23
D.1. Introducción	23
D.2. Estructura de directorios	23
D.3. Manual del programador	23

D.4. Compilación, instalación y ejecución del proyecto	23
D.5. Pruebas del sistema	23
Apéndice E Documentación de usuario	25
E.1. Introducción	25
E.2. Requisitos de usuarios	25
E.3. Instalación	25
E.4. Manual del usuario	25
Apéndice F Anexo de sostenibilización curricular	27
F.1. Introducción	27
Bibliografía	29

Índice de figuras

A.1. Sprint 1.	4
A.2. Sprint 2.	5
A.3. Sprint 3.	6
A.4. Sprint 4.	7
A.5. Sprint 5.	8
A.6. Sprint 6.	9
A.7. Sprint 7.	10
A.8. Sprint 8.	11
A.9. Sprint 9.	12
A.10.Sprint 10.	13
A.11.Sprint 11.	14

Índice de tablas

A.1. Asignación temporal de los <i>Story Points</i>	2
B.1. CU-1 Nombre del caso de uso.	19

Apéndice A

Plan de Proyecto Software

A.1. Introducción

El plan de proyecto software es el documento que define cómo se va a gestionar y ejecutar el desarrollo de un producto o sistema informático. Establece el alcance, los objetivos, el cronograma, los recursos, el presupuesto, los riesgos y los criterios de calidad, así como la organización del equipo y los mecanismos de seguimiento y control. En esencia, convierte una idea de software en un itinerario gestionable, con responsabilidades claras y procedimientos para controlar el progreso y los cambios.

Su importancia radica en que reduce la incertidumbre y aumenta la probabilidad de éxito académico y técnico. Permite justificar decisiones, anticipar problemas, y demostrar competencias en gestión de proyectos, ingeniería de requisitos, calidad y comunicación técnica.

En este documento se va a incluir la planificación temporal de las fases y tareas del proyecto. Para cada una se debe establecer una fecha de inicio y se debe estimar la fecha final. Para realizar esto hay que tener en cuenta el peso de cada tarea, los requisitos previos y las dependencias.

También, se va a valorar la viabilidad del proyecto. En concreto, la viabilidad económica y la legal. En la viabilidad económica se estiman los costes y los beneficios del proyecto. Mientras que en la viabilidad legal analizan las leyes y licencias que afecten al desarrollo.

A.2. Planificación temporal

Para la planificación se utiliza una metodología Scrum, aplicando iteraciones (*sprints*). Al finalizar cada *sprint* se realiza una reunión para la revisión del incremento y la planificación del siguiente. En la planificación se seleccionan y valoran las tareas (*issues*) a realizar en el siguiente *sprint*. Estas tareas se han valorado usando los *story points* de ZenHub. A las tareas que se subdividen en otras se les ha asignado solo los puntos extras respecto a sus subtareas, los *story points* no se han considerado acumulativos.

Para simplificar la asignación de los *story points* se les ha asignado una estimación temporal recogida en la siguiente tabla A.1:

<i>Story Points</i>	Tiempo invertido (horas)
1 <i>Story Point</i>	0,5
2 <i>Story Points</i>	1
3 <i>Story Points</i>	2
5 <i>Story Points</i>	4
8 <i>Story Points</i>	6
13 <i>Story Points</i>	9
21 <i>Story Points</i>	15
40 <i>Story Points</i>	más de 20

Tabla A.1: Asignación temporal de los *Story Points*.

Sprint 0 (11/09/2025 - 18/09/2025)

Este *sprint* marcó el comienzo del proyecto. Los objetivos fueron realizar una primera toma de contacto con los RAG y los LLM, investigando sobre qué son, cómo funcionan y qué ventajas aportan los RAG. También, se planteó empezar a escribir los conceptos teóricos relevantes y crear el repositorio **GitHub** para el proyecto. Como el *sprint* se diseñó antes de la creación del repositorio, no se creó el *milestone* asociado como se ha hecho en los *sprints* posteriores. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Crear el repositorio en GitHub.
- Investigar sobre los RAG y LLM.
 - Leer el capítulo 1 del libro: *A simple guide to retrieval augmented generation* [1].

- Leer el capítulo 2 del libro: *A simple guide to retrieval augmented generation* [1]
- Escribir conceptos teóricos sobre lo investigado.

Para realizar estas tareas se estimaron 6 horas de trabajo pero en realidad fueron 9. Al finalizar el *sprint* se completaron todas las tareas.

Sprint 1 (18/09/2025 - 25/09/2025)

Los objetivos del **Sprint 1** fueron investigar sobre los LLM y los RAG y escribir la información más relevante como conceptos teóricos en la memoria. Comenzar a escribir los objetivos del proyecto y las técnicas y herramientas definidas hasta el momento, como la metodología Scrum o el repositorio. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Escribir objetivos del proyecto y técnicas y herramientas.
- Escribir conceptos teóricos.
 - Investigar sobre los RAG y los LLM.
 - Leer el capítulo 3 del libro: *A simple guide to retrieval augmented generation* [1].
 - Leer el capítulo 4 del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 8,5 horas de trabajo pero en realidad fueron 10. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura **A.1**



Figura A.1: Sprint 1.

Sprint 2 (25/09/2025 - 02/10/2025)

Los objetivos del **Sprint 2** fueron seguir investigando sobre los LLM y RAG y explicar la información relevante en los conceptos teóricos. Investigar sobre el *Web Scraping* y explicar en los anexos el plan de proyecto, en concreto la planificación temporal y los *sprints*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir las correcciones a la memoria.
- Completar el apartado pipeline de generación del apartado conceptos teóricos.
- Investigar sobre *Web Scraping*.
 - Investigar sobre HTML.
 - Investigar sobre CSS.
- Explicar el plan de proyectos en los anexos.

Para realizar estas tareas se estimaron 10 horas de trabajo pero en realidad fueron 12. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura [A.2](#)

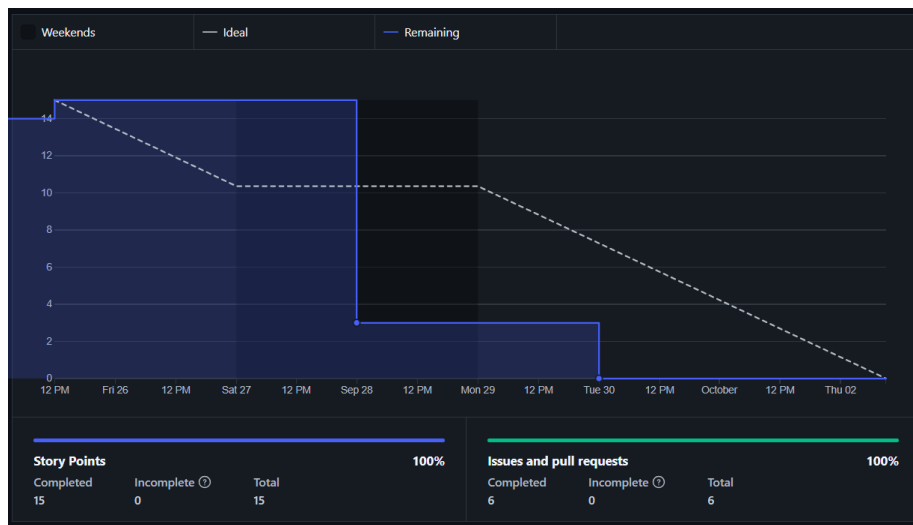


Figura A.2: Sprint 2.

Sprint 3 (2/10/2025 - 09/10/2025)

Los objetivos del **Sprint 3** fueron realizar un primer prototipo de *Web Scraping*, tanto en *Selenium* como en *Playwright*. Incluir en la memoria las correcciones y la información relevante. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir las correcciones a la memoria.
- Realización del comienzo de la aplicación de *web scraping*.
 - Investigar *web scraping* con Selenium.
 - Investigar *web scraping* con Playwright.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 5 del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.3



Figura A.3: Sprint 3.

Sprint 4 (9/10/2025 - 16/10/2025)

Los objetivos del **Sprint 4** fueron continuar realizando la aplicación de *Web Scraping*, tanto en *Selenium* como en *Playwright*, para sacar licitaciones de la página de contratación del sector público. Incluir en la memoria una comparación entre ambas aplicaciones de *Web Scraping*. Seguir investigando sobre los RAG. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Continuar con el *script* de *web scraping* usando Playwright.
 - Continuar con el *script* de *web scraping* usando Selenium.
 - Estudiar sobre json.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 6 del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 14 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.4

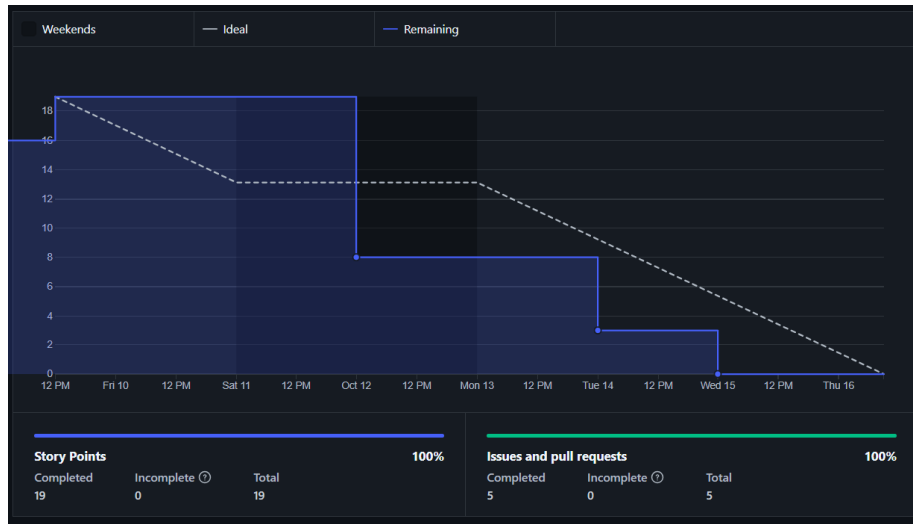


Figura A.4: Sprint 4.

Sprint 5 (16/10/2025 - 23/10/2025)

Los objetivos del **Sprint 5** fueron investigar sobre las mejoras que aporta la *API* asíncrona de *Playwright* y como se implementa. También, se planificado investigar sobre la gestión de dependencias y entornos en *Python*. Todo esto mientras se sigue investigando sobre los *RAG* y *LLM* y se escriben los aspectos más relevantes en la memoria. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Realizar código con *Playwright* asíncrono.
 - Investigar sobre la *API* asíncrona de *Playwright*.
- Investigar sobre las dependencias y los entornos en *Python*.
- Escribir conceptos teóricos en la memoria.
 - Leer más capítulo del libro: *A simple guide to retrieval augmented generation* [1].

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18, ya que se tuvo que reescribir parte del código realizado en los *sprints* anteriores para el *Web Scraping* debido a una actualización en la página de **Contratación del Sector Público**. Este cambio afectó en mayor medida a la extracción de información de las licitaciones, ya que ha sido la página que más ha cambiado con dicha actualización. A pesar de este contratiempo al finalizar el *sprint* se completaron todas las tareas.



Figura A.5: Sprint 5.

El *burndown* de este *sprint* se ilustra en la figura A.5

Sprint 6 (23/10/2025 - 30/10/2025)

Los objetivos del **Sprint 6** fueron descargar los pdfs de las licitaciones de la página de **Contratación del Sector Público**. Para descargar dichos pdfs se va a usar de base el programa de *Web Scraping* del *sprint* anterior. Además, se han fijado como objetivos investigar sobre los modelos de inteligencia artificial de *Hugging Face* e instalar y probar el funcionamiento de *Ollama*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre *Ollama*.
- Investigar y probar modelos de inteligencia artificial de *Hugging Face*.
- Investigar sobre formas de implementar los RAG.
- Descargar los pdfs de las licitaciones de la la página de **Contratación del Sector Público**.
- Instalar las dependencias en *Poetry*.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15, se ha usado un servidor de la UBU para descargar un zip con las licitaciones. Para acceder a dichas licitaciones se ha usado el archivo "*resultados_playwright_asincrono*" obtenido en el *sprint* anterior, en el cual se almacenan los enlaces a los pdfs de cada licitación. Al finalizar el *sprint* se completaron todas las tareas.



Figura A.6: Sprint 6.

El *burndown* de este *sprint* se ilustra en la figura A.6

Sprint 7 (30/10/2025 - 6/11/2025)

El objetivo principal del **Sprint 7** fue investigar e implementar distintos tipos de *tokenize*. Esto se planteó como un primer paso para la implantación total del RAG. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre los *tokenize*.
- Implementar distintos tipos de *tokenize*.
- Comparar las implementaciones de *tokenize*.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15. En este *sprint* se han llevado a cabo todas las tareas planificadas, pero sería destacable que en los *scripts* de las diversas pruebas de *tokenizers* no solo se han incluido estos. Si no que también se han incluido funciones para probar su uso, como pueden ser *propts* y funciones para dividir los pdfs de las licitaciones.

El *burndown* de este *sprint* se ilustra en la figura A.7



Figura A.7: Sprint 7.

Sprint 8 (6/11/2025 - 13/11/2025)

El objetivo principal del **Sprint 8** fue implementar una base de datos para el RAG y juntarlo con uno de los *tokenizers* ya implementados del *sprint* anterior. Además, en este *sprint* se planteó como objetivo obtener los pliegos definitivos de de la página de **Contratación del Sector Público**.

Esto se debe a que los pliegos obtenidos anteriormente no eran correctos, se trataban de resúmenes incompletos que no servían para alimentar al RAG. Durante el *sprint* 7, se encontraron los pliegos correctos y se fijó como objetivo del siguiente *sprint* modificar los programa de *Web Scraping* para descargarlos. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Obtener pliegos correctos.
- Estudiar sobre las distintas bases de datos.
- Implementar la base de datos.
- Explicar la comparativa de las bases de datos.

Para realizar estas tareas se estimaron 12 horas de trabajo, pero en realidad fueron 16. En este *sprint* se han llevado a cabo todas las tareas planificadas. Se ha implementado la base de datos vectorial *Qdranty* y se ha juntado con el diseño de *tokenizer* del *sprint* anterior, aunque se ha cambiado el modelo para usar el del libro **LLM Engineer's Handbook**. Además, se



Figura A.8: Sprint 8.

han realizado los *embeddings* para poder almacenarlos en la base de datos vectorial.

El *burndown* de este *sprint8* se ilustra en la figura [A.8](#)

Sprint 9 (13/11/2025 - 20/11/2025)

El objetivo principal del **Sprint 9** fue realizar un *script* que, dada una consulta, devuelva el *chunk* más parecido. Junto con este *chunk* debe devolver el nombre y el título del pdf, así como una respuesta del LLM basándose en dicho *chunk*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Implementar funciones que devuelvan el nombre del pdf al que pertenece el *chunk*.
- Mejorar la partición de los *chunks* añadiendo solapamiento (*overlap*).
- Obtención del fragmento del *chunk*.
- Implementar código para que el LLM de una respuesta a partir de una pregunta del usuario.
- Añadir al código funciones para medir los tiempos
- Usar loggers en el código

Para realizar estas tareas se estimaron 18 horas y más o menos se ha cumplido dicha estimación. En este *sprint* se han llevado a cabo todas las

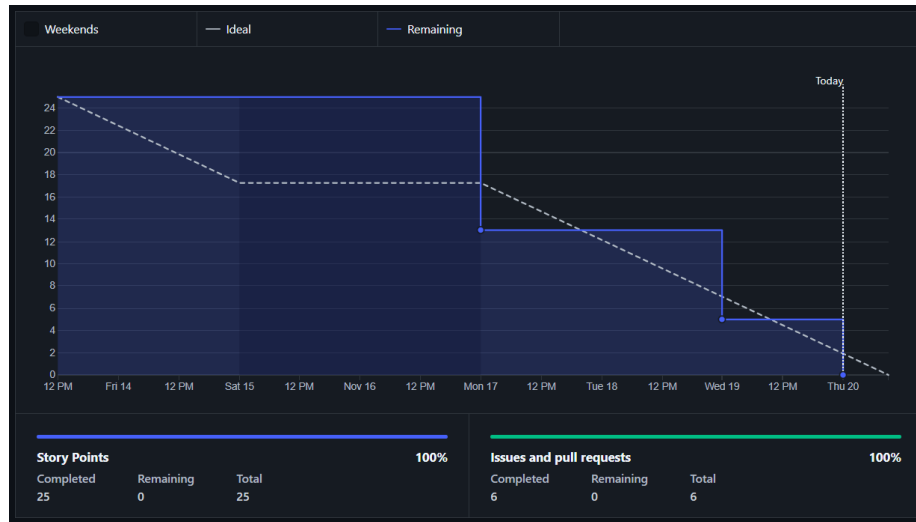


Figura A.9: Sprint 9.

tareas planificadas. Se ha mejorado el código añadiendo medidas de los tiempos de ejecución e implementando el uso de *loggers*. También, se ha añadido solapamiento en los *chunks* lo que mejora la eficacia de estos. Por otro lado se ha implementado el objetivo principal, es decir, el *script* que permite que el usuario realice una pregunta y el LLM conteste basándose en el *chunk* más parecido a la pregunta. Además se indica el pdf del que ha sacado la información así como el *chunk* concreto.

El *burndown* de este *sprint9* se ilustra en la figura [A.9](#)

Sprint 10 (20/11/2025 - 27/11/2025)

El objetivo principal del [Sprint 10](#) fue intentar convertir los pdfs a *Markdown*. Para ello se han valorado diversas formas de realizar la conversión, por ejemplo, desde un *OCR* vinculado a un *LLM* o directamente empleando una biblioteca de *Python*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Buscar formas de automatizar la conversión a *Markdown*.
- Realizar la conversión de un pdf a *Markdown*.
 - Automatizar la conversión a *Markdown*.

Para realizar estas tareas se estimaron 10 horas, pero debido a la gran heterogeneidad de formatos de los pdfs fueron más, aproximadamente unas



Figura A.10: Sprint 10.

15. A pesar de que se han obtenido resultados significativamente mejores que en las primeras aproximaciones no se ha conseguido ningún *Markdown* lo suficiente mente bueno. Esto se debe a que cada pdf de licitaciones tiene un formato distinto y no se ha conseguido ningún *Script* que detecte correctamente todas las secciones.

El *burndown* de este *sprint10* se ilustra en la figura A.10

***Sprint 11* (27/11/2025 - 4/12/2025)**

El objetivo principal del **Sprint 11** fue intentar implementar un *Script* que basándose en los resultados obtenidos del anterior sea capaz de detectar correctamente las secciones de los pdfs. Además, se propuso comenzar a investigar sobre el funcionamiento de *Flask*. Ya que esta aplicación se va a utilizar en el proyecto para realizar una web para realizar las consultas al LLM. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Probar nuevos OCRs para la conversión de los pdfs a *Markdown*.
- Investigar sobre *Flask*.

Para realizar estas tareas se estimaron 8 horas. AL final se emplearon cerca de 12 horas debido al mismo problema del *Sprint* anterior, que no se ha conseguido implementar un *script* capaz de detectar correctamente todas las secciones.



Figura A.11: Sprint 11.

El *burndown* de este *sprint11* se ilustra en la figura [A.11](#)

Sprint 12 (4/12/2025 - /12/2025)

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Una muestra de cómo podría ser una tabla de casos de uso:

B.2. Objetivos generales

B.3. Catálogo de requisitos

Requisitos funcionales

- RF-1 Inserción de documentos: el sistema debe permitir cargar documentos de licitaciones con el objetivo de construir y actualizar la base de datos vectorial.
- RF.2 Procesado de los documentos: el sistema debe procesar de manera automática los documentos introducidos para prepararlos para que el RAG pueda usarlos.
 - RF-2.1 Extracción de información: el sistema debe extraer automáticamente el contenido textual de los documentos introducidos, ignorando páginas o documentos sin texto extraíble.
 - RF-2.2 Conversión del contenido: el sistema debe transformar el texto extraído a un formato homogéneo, concretamente a *markdown* para que sea más fácil su procesamiento.
 - RF-2.2.1 Conservar estructura básica: el sistema debe conservar la estructura básica del texto siempre que sea posible (secciones, títulos, saltos de línea, párrafos, etc).

- RF-2.3 Limpieza y normalización del texto: el sistema debe limpiar y normalizar el texto que se extrae de los documentos.
 - RF-2.3.1 Eliminar caracteres: el sistema debe eliminar los caracteres irrelevantes o problemáticos.
 - RF-2.3.2 Normalización: el sistema debe normalizar los espacios, saltos de línea y formatos inconsistentes.
- RF-3 Segmentación en chunks: el sistema debe dividir los documentos ya procesados en fragmentos de texto (*chunks*) adecuados para el modelo de *embeddings*.
 - RF-3.1 Tamaño: el sistema debe segmentar el texto respetando un máximo de tamaño por *token* para evitar errores.
 - RF-3.2 Solapamiento: el sistema debe aplicar solapamiento entre *chunks* para preservar el contexto.
 - RF-3.3 Contexto: cada *chunk* debe tener una referencia al documento original y a su posición dentro de él.
- RF-4 Generación de *embeddings*: el sistema debe generar representaciones vectoriales (*embeddings*) para cada *chunk*.
- RF-5 Persistencia en base de datos vectorial: el sistema debe almacenar los *chunks* y sus *embeddings* en una base de datos vectorial.
 - RF-5.1 Almacenar chunks: el sistema debe almacenar el contenido textual de cada *chunk*.
 - RF-5.2 Almacenar embeddings: el sistema debe almacenar el *embedding* asociado a cada *chunk*.
 - RF-5.3 Almacenar metadatos: el sistema debe almacenar metadatos relevantes como el nombre del archivo, el título y el segmento incluido en el chunk.
 - RF-5.4 Actualización: el sistema debe permitir la actualización o recreación de la colección vectorial en el caso de incluir documentos nuevos.
- RF-6 Recuperación de información por similitud: el sistema debe recuperar información relevante a partir de las consultas de los usuarios.
 - RF-6.1 Generación de *embedding*: el sistema debe generar el *embedding* de la consulta del usuario.
 - RF-6.2 Búsqueda por similitud: el sistema debe realizar una búsqueda por similitud en la base de datos vectorial.
- RF-7 Construcción del *prompt* con contexto: el sistema debe construir un *prompt*, para alimentar al LLM, que incluya los *chunks* recuperados, la pregunta del usuario y reglas de uso del contexto.

- RF-8 Generación de respuestas mediante un LLM: el sistema debe enviar el *prompt* generado a un modelo LLM y mostrar la respuesta de dicho modelo al usuario.
 - RF-8.1 Evitar alucinaciones: El sistema no debe responder cuando no haya contexto relevante suficiente.
- RF-9 Trazabilidad de respuesta: el sistema debe devolver junto con la respuesta los metadatos del documento origen y los *chunks* empleados para la generación de la respuesta.
- RF-10 Gestión de usuarios: el sistema debe permitir la gestión de usuarios a través de la aplicación web.
 - RF-10.1 Registros y autenticación: el sistema debe permitir el registro y autenticación de los usuarios.
 - RF-10.2 Sesiones: el sistema debe gestionar las sesiones de los usuarios de forma segura.
 - RF-10.3 Roles: el sistema debe permitir asignar a los usuarios el rol de administradores o usuarios normales.
 - RF-10.4 Administradores: los administradores deben poder añadir usuarios, eliminarlos y cambiar su rol.
- RF-11 Interfaz de consultas: los usuarios autenticados podrán realizar consultas y ver las respuestas del sistema.
 - RF-11.1 Historial de consultas: los usuarios podrán ver su historial de consultas.

Requisitos no funcionales

- RNF-1 Reproducibilidad: el proyecto debe ejecutarse en un entorno reproducible.
- RNF-2 Rendimiento: el sistema debe ser capaz de procesar los documentos sin intervención manual. Además, el sistema tiene que responder en un tiempo razonable.
- RNF-3 Robustez ante errores: el sistema debe continuar ejecutándose ante fallos esperables en datos, red o servicios externos.
 - RNF-3.1 Documentos corruptos: el sistema debe saltarse los documentos corruptos o sin texto extraíble y continuar con el resto de documentos.
 - RNF-3.2 *Timeouts*: el sistema debe manejar los *timeouts* del LLM devolviendo mensajes controlados al usuario.
- RNF-4 Trazabilidad técnica: el sistema debe realizar *logs* de carga de modelo, lectura de PDFs, *chunking*, *embeddings*, guardado y consulta.

- RNF-5 Seguridad: el sistema debe gestionar de forma segura la información sensible de los usuarios y el acceso a la aplicación.
 - RNF-5.1 Contraseñas: las contraseñas deben almacenarse *hasheadas*.
 - RNF-5.2 Control de acceso por roles: las funciones administrativas deben estar protegidas por rol para que solo las pueda realizar el administrador.
 - RNF-5.3 Protección de secretos: las claves sensibles no deben estar públicas en el repositorio.
- RNF-6 Usabilidad: la interfaz debe permitir a los usuarios realizar las consultas y ver el historial de manera sencilla. Además, los textos deben ser legibles.
- RNF-7 Calidad del *software*: debe existir un conjunto de pruebas que asegure el correcto funcionamiento del programa.
- RNF-8 Disponibilidad: la web debe garantizar un nivel de disponibilidad adecuado que permita a los usuarios acceder a la aplicación y realizar consultas.
- RNF-9 Recuperación y tolerancia a fallos: el sistema debe disponer de mecanismos que permitan la recuperación de información y la restauración del servicio tras fallos, garantizando la integridad de los datos.

B.4. Especificación de requisitos

Actores

Los actores del sistema serán:

- **Usuario anónimo:** usuario que no ha iniciado sesión (solo puede registrarse y autenticarse)
- **Usuario autenticado:** usuario que puede acceder a la aplicación sus acciones en al aplicación cambiarán dependiendo de su rol.
 - **Usuario normal:** usuario autenticado que puede consultar el RAG y ver su historial
 - **Usuario administrador:** usuario autenticado con permisos de gestión de usuarios y de carga y actualización documental
- **Sistema:** es el encargado del procesamiento, *chunking*, *embeddings*, guardado y consulta.

CU-1	Ejemplo de caso de uso
Versión	1.0
Autor	Alumno
Requisitos asociados	RF-xx, RF-xx
Descripción	La descripción del CU
Precondición	Precondiciones (podría haber más de una)
Acciones	1. Pasos del CU 2. Pasos del CU (añadir tantos como sean necesarios)
Postcondición	Postcondiciones (podría haber más de una)
Excepciones	Excepciones
Importancia	Alta o Media o Baja...

Tabla B.1: CU-1 Nombre del caso de uso.

Apéndice C

Especificación de diseño

- C.1. Introducción
- C.2. Diseño de datos
- C.3. Diseño arquitectónico
- C.4. Diseño procedimental

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía

- [1] Abhinav Kimothi. *A Simple Guide to Retrieval Augmented Generation*. Manning Publications, July 2025. [Consultado 8-Septiembre-2025 a 26-Septiembre-2025].